

Technical Execution Blueprint: Production-Grade Artificial Intelligence Implementation and Model Adaptation

The following blueprint delineates a rigorous, multi-phase execution framework designed to elevate an intermediate engineering skill set to a production-ready standard within the domain of artificial intelligence. The directives are strictly formulated to enforce independent problem-solving, architectural comprehension, and disciplined debugging. The operational constraints dictate the use of limited compute environments, specifically cloud-based instances such as Google Colab, necessitating highly optimized, memory-conscious engineering practices. The framework is bifurcated into two primary technical achievements: the ground-up implementation of a complex research architecture and the parameter-efficient adaptation of a Large Language Model (LLM) utilizing highly structured enterprise data.

Phase 1: Research Paper Deconstruction and Implementation

The objective of the initial phase is to isolate, deconstruct, and implement a seminal artificial intelligence research paper within a stringent two-to-four-week timeline. The implementation must yield a functional, minimal working system built from rudimentary tensor operations and core libraries, rather than relying on high-level abstractions that obscure the underlying mechanics. The selected focus areas are constrained to Retrieval-Augmented Generation (RAG), Efficient Transformers, or Lightweight LLM pipelines.

Candidate Architecture Analysis

The practitioner is required to select one of the following three architectures for manual implementation. Each candidate has been vetted for its reliance on open datasets, its profound industry relevance, and its capacity to challenge existing engineering paradigms without requiring distributed computing clusters.

Candidate 1: Self-Reflective Retrieval-Augmented Generation (Self-RAG)

Conventional Retrieval-Augmented Generation frameworks suffer from a rigid, unconditional retrieval mechanism. These pipelines indiscriminately fetch external documents regardless of whether the specific query demands factual grounding, which frequently leads to context window pollution, diminished model versatility, and the generation of unhelpful or hallucinated responses.¹ The Self-RAG architecture completely overhauls this paradigm by introducing an end-to-end framework that trains a single, arbitrary LLM to adaptively retrieve passages on demand and, crucially, to critique its own generations.¹

The core problem statement addresses the factual inaccuracies and obsolescence of

parametric knowledge within large language models, while simultaneously solving the inefficiency of naive RAG.¹ The architecture functions through the injection of specialized reflection tokens directly into the model's vocabulary and generation stream. These tokens act as control mechanisms during inference, allowing the system to dynamically alter its behavior based on the specific requirements of the current task.¹

The data pipeline for this architecture requires a specialized instruction dataset containing these embedded reflection tokens. The original researchers have open-sourced the selfrag_train_data repository, which contains the necessary formatted interactions.³ The model architecture itself does not require structural modifications to the Transformer blocks; instead, it relies on an expanded tokenizer and a highly modified inference loop. The system utilizes four primary reflection tokens⁵:

- The Retrieve token, which prompts the system to trigger a parallel retrieval process.
- The ISREL token, which critiques whether the retrieved document provides useful, relevant information.
- The ISSUP token, which evaluates if the model's generated text is fully supported, partially supported, or entirely unsupported by the retrieved passage.
- The ISUSE token, which rates the overall utility of the final response.

Implementing the inference loop requires abandoning standard generation functions. The practitioner must construct a custom decoding loop that processes an input prompt and dynamically predicts the Retrieve token. If the probability of retrieval surpasses a designated threshold, the algorithm must pause generation, query a retriever model or vector database on demand, and concurrently process multiple retrieved passages in parallel.⁵ The model then generates text segments alongside critique tokens, which are used as a scoring mechanism to rank the multiple parallel outputs and select the most factually grounded response.⁶ Evaluation of this system is rigorously conducted using fact verification datasets and open-domain question answering benchmarks, such as PubMedQA, to measure the reduction in hallucination rates.⁷

Candidate 2: Fast Inference from Transformers via Speculative Decoding

The physical limitations of modern hardware accelerators, specifically the memory bandwidth bottleneck of GPUs, present a massive hurdle for autoregressive language models. During standard generation, decoding a sequence of length K requires K independent, serial forward passes of the entire model.⁹ For models containing billions of parameters, loading the immense weight matrices from High Bandwidth Memory (HBM) to the compute cores for a single token computation leaves the parallel processing capabilities of the GPU severely underutilized.¹¹ Speculative decoding directly addresses this hardware inefficiency by computing several tokens in parallel without altering the underlying probability distribution of the outputs.⁹

The architectural solution mandates a dual-model configuration: a massive, highly capable

target model (e.g., Mistral 7B) and a highly efficient, lightweight draft model (e.g., a 60M to 100M parameter approximation model).¹¹ The logic relies on the observation that numerous tokens within a sequence are relatively easy to predict, requiring minimal computational power, while only complex reasoning steps strictly necessitate the massive parameter count of the target model.¹¹

The implementation of the inference loop—the core of this project—requires a sophisticated algorithm based on speculative execution and rejection sampling.¹¹ The draft model

autoregressively generates a sequence of γ draft tokens. Subsequently, the target model processes this entire drafted sequence in a single parallel forward pass, utilizing its superior capacity to calculate the true probability distribution for each of the γ tokens simultaneously.¹¹

The practitioner must implement the exact mathematical rejection sampling mechanism. If the draft model's probability for a token is $q(x)$ and the target model's true probability is $p(x)$, the token is accepted if a random uniform variable is less than the ratio of $p(x)$ to $q(x)$.¹¹ If the token is rejected, the drafted sequence is immediately truncated, the correct token is sampled directly from the target model's distribution, and the draft model is reset.¹¹ The critical engineering challenge is ensuring that the computational overhead of running the draft model and the target model together does not exceed the time saved by the parallel verification.¹⁶ Evaluation is strictly empirical: the output must remain mathematically identical to standard decoding, while the Tokens Per Second (TPS) metric must demonstrate a measurable 2X to 3X acceleration.⁹

Candidate 3: Precise Zero-Shot Dense Retrieval with Hypothetical Document Embeddings (HyDE)

Information retrieval within standard RAG pipelines typically relies on contrastive encoders mapping queries and documents into a shared vector space. However, this semantic matching frequently fails in zero-shot scenarios where domain-specific labeled data is unavailable, as the semantic gap between a terse user query and a detailed, expansive source document is too vast for simple inner-product similarity to bridge.¹⁸ The Hypothetical Document Embeddings (HyDE) architecture solves this by fundamentally decoupling relevance modeling from representation learning.²⁰

The pipeline requires two distinct models acting in sequence: an instruction-tuned generative LLM and an unsupervised contrastive encoder.¹⁸ The process begins when the generative model is prompted with the user's query and instructed to write a detailed, hypothetical document that answers the question.¹⁸ This generated text is entirely synthetic; it may contain severe factual errors or hallucinations, but it perfectly mimics the structural and linguistic "relevance patterns" of the target document.¹⁸

The data pipeline then takes this hypothetical document and passes it through the contrastive

encoder (such as Contriever or an equivalent modern embedding model). The dense bottleneck of the encoder acts as a lossy compressor, filtering out the hallucinated specifics while retaining the core semantic intent.²⁰ The resulting vector is then used to perform a standard vector database search against a corpus of real documents.¹⁸ By querying the vector space with an embedding of a document rather than an embedding of a question, the system searches within a localized neighborhood of documents, drastically increasing zero-shot precision.²⁰ The implementation is highly modular and requires the practitioner to build the generation, embedding, and retrieval pipelines, subsequently evaluating the system against standard BM25 baselines using datasets such as TREC DL19 or BEIR.²⁰

Execution Schedule: Phase 1 Mandatory Directives

The practitioner must select one of the three architectures immediately. For the purpose of establishing a rigorous baseline, the following directives assume the selection of **Speculative Decoding**, as it presents the most demanding engineering constraints regarding tensor manipulation and algorithmic precision.

Milestone Phase	Objective Focus	Output Requirement
Days 1-3	Algorithmic Deconstruction	Mathematical translation to PyTorch pseudo-code.
Days 4-7	Naive Pipeline Construction	A functional, unoptimized forward pass mechanism.
Days 8-10	Rejection Sampling Implementation	Mathematically sound token acceptance/rejection logic.
Days 11-14	Benchmarking and Refactoring	Modular codebase yielding measurable TPS acceleration.

Days 1-3: Theoretical Foundation and Environment Initialization

The initial stage forbids the writing of implementation code. The practitioner must achieve complete theoretical mastery over the architecture.

- **Task 1:** Isolate the core mathematical formulas governing the architecture. For Speculative Decoding, this requires mapping the probability distributions $p(x)$ and $q(x)$ to specific tensor outputs from the model's language modeling head.
- **Task 2:** Establish the compute environment. Verify CUDA availability, initialize the virtual environment, and install base requirements (PyTorch, Transformers, Accelerate).
- **Task 3:** Select the target and draft models. The draft model must possess a vocabulary identical to the target model to prevent tensor shape mismatches during probability comparisons. A combination such as the Gemma 7B target and Gemma 2B draft, or TinyLlama variants, is mandated.¹²
- **Comprehension Checkpoint:** The practitioner must articulate precisely why a rejected token in Speculative Decoding still permits the generation of one valid, mathematically correct token during the verification step without requiring an additional forward pass.

Days 4-7: The Naive Architecture Implementation

The objective is to force the system to compile and execute a complete cycle, regardless of severe inefficiencies or suboptimal memory management.

- **Task 1:** Construct a custom decoding loop, entirely bypassing the native `model.generate()` abstractions provided by high-level libraries.
- **Task 2:** Implement the draft generation phase. Write the autoregressive loop that forces the small model to predict a fixed number of tokens (γ , the acceptance length) ahead of the current state.¹²
- **Task 3:** Implement the parallel verification phase. Feed the concatenated sequence of the original prompt and the draft tokens into the target model in a single forward pass, extracting the logit tensors for the sequence.¹⁴
- **Debugging Checkpoint:** Ensure that the Key-Value (KV) cache is appropriately managed. Naively passing the entire sequence through the models at every step will result in geometric memory growth and an immediate Out Of Memory (OOM) fatal error.
- **Comprehension Checkpoint:** The practitioner must identify the exact tensor dimension that represents the probability distribution across the vocabulary space and articulate how the softmax function is applied to the raw logits.

Days 8-10: Mathematical Rigor and Probability Sampling

With a crude pipeline established, the practitioner must implement the strict mathematical bounds that guarantee output equivalence.

- **Task 1:** Implement the rejection sampling logic. Construct the comparative algorithm that evaluates the target probabilities against the draft probabilities.
- **Task 2:** Implement the fallback mechanism. When a token is rejected, the logic must discard all subsequent drafted tokens in the current γ sequence, sample the corrected

token from the modified target distribution, and append it to the final output.¹¹

- **Task 3:** Address temperature scaling. Ensure that sampling parameters (temperature, top-p, top-k) are uniformly applied to the logits of both models prior to the calculation of the final probabilities. Failure to synchronize these parameters will result in catastrophic divergence and a near-zero acceptance rate.

Days 11-14: Optimization, Profiling, and Measurement

The final stage of Phase 1 demands strict engineering optimization. The pipeline must demonstrate empirical superiority over the baseline.

- **Task 1:** Establish the baseline control metric. Execute a standard autoregressive generation loop using only the target model and record the Tokens Per Second (TPS) across a standardized set of prompts.¹⁴
- **Task 2:** Profile the Speculative Decoding pipeline. Record the acceptance rate of the draft tokens.
- **Debugging Checkpoint:** If the overall generation speed is slower than the baseline control, the acceptance rate is insufficient to offset the computational overhead of the dual-model architecture.¹⁶ The practitioner must systematically adjust the γ parameter or investigate potential misalignment in the tokenization process.
- **Comprehension Checkpoint:** The practitioner must define the theoretical maximum speedup possible given a specific γ value, and explain how memory bandwidth constraints on the specific cloud hardware dictate this upper bound.

Phase 2: Parameter-Efficient Fine-Tuning and Enterprise Adaptation

Phase 2 pivots from architectural engineering to data-centric model adaptation. The objective is to utilize Quantized Low-Rank Adaptation (QLoRA) to fine-tune an open-weight LLM on a complex, highly practical dataset. To maximize commercial relevance and ensure the development of business-ready skills, the selected domain is an Enterprise Resource Planning (ERP) and Business Operations Assistant. This specific domain circumvents the oversaturation of generic chatbots and directly addresses the urgent industry requirement for dynamic intelligence platforms capable of interpreting complex supply chain metrics, financial transactions, and multi-table customer records.²³

Enterprise Dataset Sourcing and Structural Analysis

The fine-tuning protocol strictly mandates the use of real-world or high-fidelity enterprise data. Training a model on rudimentary, synthetic prompt-response pairs is insufficient for production-grade engineering. The primary data source for this phase is the SAP SALT (Sales Autocompletion Linked Business Tables) dataset, augmented by the Bitext Customer Support

Dataset.²⁵

The SAP SALT dataset represents a foundational challenge in artificial intelligence: the adaptation of sequential language models to structured, tabular data.²⁶ The dataset contains anonymized data extracted directly from a commercial ERP system, specifically SAP Sales and Distribution modules.²⁶ The critical engineering hurdle is that business data is inherently relational; a single sales order entry is intrinsically linked via foreign keys to disparate supplier databases, inventory matrices, and customer profiles.²⁷

Furthermore, the data is highly heterogeneous, containing a volatile mix of categorical text fields and continuous numerical values, coupled with extreme distribution imbalances where specific product categories dominate the transaction logs.²⁹ To instill conversational fluidity and intent recognition, the Bitext dataset provides over 26,000 multi-turn instruction pairs spanning 27 distinct customer service intents, such as order cancellations, invoice validation, and product compatibility inquiries.²⁵

Data Pipeline Engineering and Serialization

Providing raw CSV files or SQL database dumps to an LLM during the fine-tuning phase will result in catastrophic failure. The practitioner must engineer a robust serialization pipeline to transform relational database architectures into coherent, natural language instruction formats suitable for causal language modeling.

- **Task 1: Relational Linearization:** The multi-table connections within the SAP SALT dataset must be systematically flattened into contextual strings. The practitioner must write a script that merges tables based on foreign keys and translates the row data into a serialized format. A relational join spanning an inventory table and a sales table must be transformed into a unified text block that explicitly defines the system context, the raw input variables, and the target instruction.
- **Task 2: Multi-Turn State Construction:** Enterprise queries are rarely resolved in a single interaction. The data must be structured to support multi-turn conversational state tracking. This requires formatting the data such that the state s_t encompasses the entire dialogue history up to the current turn t , forcing the model to learn context retention and temporal reasoning across multiple interactions.³¹
- **Task 3: Chat Template Enforcement:** The serialization script must output the data in a strict JSONL format that aligns perfectly with the chat template of the chosen base model (e.g., Mistral's specific instruction tags). Every training example must be an array of messages, explicitly defining the role (system, user, assistant) and the corresponding content.³²

The QLoRA Fine-Tuning Architecture

Due to the strict compute constraints inherent in utilizing cloud environments like Google Colab, full-parameter fine-tuning of billions of parameters is mathematically impossible due to

extreme GPU VRAM limitations. The architecture dictates the utilization of QLoRA (Quantized Low-Rank Adaptation) on a highly capable base model, such as Mistral 7B or Phi-2.¹⁴

The mathematical intuition behind LoRA relies on the hypothesis that the change in weights during specific task adaptation possesses a low "intrinsic rank." During standard backpropagation, a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ receives a dense gradient update ΔW . LoRA constrains this update by representing it as the product of two much smaller, low-rank matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, resulting in the formula $W = W_0 + BA$, where the rank $r \ll \min(d, k)$.

QLoRA advances this by aggressively quantizing the massive frozen base model W_0 into a 4-bit NormalFloat precision format, utilizing double quantization to compress the quantization constants themselves. Only the small A and B adapter matrices are maintained in 16-bit brain float precision and receive gradient updates. This technique reduces the memory footprint by orders of magnitude while preserving near-full-parameter performance levels.

Execution Schedule: Phase 2 Mandatory Directives

Milestone Phase	Objective Focus	Output Requirement
Days 15-17	Data Linearization	Formatted JSONL instruction dataset representing relational ERP data.
Days 18-20	Architecture Initialization	Base model loaded in 4-bit precision with attached LoRA adapters.
Days 21-24	Training Loop Execution	Successful completion of fine-tuning epochs without memory overflow.
Days 25-28	Evaluation and Benchmarking	Empirical analysis of domain

		adaptation and hallucination reduction.
--	--	---

Days 15-17: Data Transformation and Quality Assurance

- **Task 1:** Download and sanitize the SAP SALT and Bitext datasets.³⁴ Construct the Python serialization script to parse the tabular structures and merge the relational data.
- **Task 2:** Apply the specific tokenization chat template. Ensure that the loss is only calculated on the assistant's responses by appropriately masking the instruction and user input tokens during the creation of the labels tensor.
- **Debugging Checkpoint:** Inspect the tokenized output manually. If the tokenization script injects extraneous special tokens or fails to append the end-of-sequence (EOS) token, the model will generate infinite, incoherent loops during inference.
- **Comprehension Checkpoint:** The practitioner must explain why training an LLM on raw CSV data formats is highly inefficient compared to serialized, natural language representations of the same tabular data.

Days 18-20: Memory Optimization and Adapter Configuration

- **Task 1:** Initialize the base model utilizing the BitsAndBytes configuration, ensuring 4-bit precision and bfloat16 compute types are active.
- **Task 2:** Configure the LoRA adapters. Identify the specific linear layers within the Transformer blocks to target. While targeting solely the query and value attention projections is a standard baseline, the complexity of multi-table ERP reasoning dictates that the practitioner should inject adapters into all linear layers, including the Multilayer Perceptron (MLP) up and down projections.
- **Task 3:** Define the hyperparameter matrix. Select an appropriate rank r (e.g., 32 or 64) and set the scaling factor α to $2r$.

Days 21-24: The Training Loop and VRAM Management

- **Task 1:** Initialize the training loop. Implement a cosine decay learning rate scheduler with a linear warmup phase to prevent the initial volatile gradients from permanently distorting the pre-trained representations.
- **Task 2:** Enable gradient checkpointing. This technique trades computational cycles for memory savings by dropping intermediate activations during the forward pass and recalculating them on demand during the backward pass.
- **Debugging Checkpoint:** If a CUDA Out of Memory error occurs during the backward pass, the practitioner must not instinctively reduce the batch size to 1. They must implement gradient accumulation steps to simulate a larger effective batch size and utilize paged optimizers (e.g., paged_adamw_32bit) to offload optimizer states to the

system CPU RAM.

- **Comprehension Checkpoint:** The practitioner must articulate the exact flow of a gradient during the backward pass in a QLoRA setup, specifying which matrices are frozen and which receive the weight updates.

Days 25-28: Evaluation and Domain Verification

Standard autoregressive loss curves provide minimal insight into actual model utility. The practitioner must implement robust evaluation protocols to verify that the ERP Assistant has achieved true domain adaptation.

- **Task 1:** Implement qualitative evaluation matrices. Generate responses to a held-out test set of complex sales order queries. The model must accurately retrieve the serialized tabular data without hallucinating non-existent inventory batch numbers.
- **Task 2:** Evaluate multi-turn state tracking. Inject a multi-turn conversation and verify that the model successfully retains context from turn $t - 2$ to execute a command at turn t .
- **Debugging Checkpoint:** If the fine-tuned model begins failing at basic conversational syntax while perfectly reciting inventory data, it has suffered from catastrophic forgetting. The practitioner must debug this by lowering the learning rate, reducing the rank r , or mixing a small percentage of generic instruction data back into the ERP dataset to serve as a regularization anchor.

Phase 3: Production Polish, Deployment, and Portfolio Integration

An optimized inference algorithm and a highly accurate fine-tuned model hold zero commercial value if they cannot be interacted with or understood by external stakeholders. Phase 3 mandates the synthesis of the prior weeks into deployable, business-ready artifacts. The final week is dedicated to interface construction, stringent repository organization, and technical documentation.

Interface Architecture and Demonstration Engineering

The practitioner must encapsulate the fine-tuned ERP Assistant and the Phase 1 inference algorithm within an interactive web application utilizing frameworks such as Gradio or Streamlit.

- **Task 1:** Construct a stateful user interface capable of maintaining multi-turn conversational history. The backend must properly format the incoming user queries through the identical chat template utilized during the Phase 2 training loop.
- **Task 2:** Expose the generation parameters to the user interface. Implement slider controls for Temperature, Top-p, and Maximum New Tokens, allowing real-time

modulation of the model's output variance.

- **Task 3:** Integrate the Phase 1 architectural mechanisms directly into the visual interface. If Speculative Decoding was implemented, the interface must feature a toggle switch to enable or disable the draft model. Crucially, it must display a real-time telemetry graph plotting the Tokens-Per-Second (TPS) latency, providing immediate visual proof of the 2X-3X hardware acceleration.⁹ If Self-RAG was implemented, the application must intercept the hidden reflection tokens (, , [Fully supported]) and pipe them into a collapsible "Reasoning Trace" sidebar. This visually demonstrates to the user that the model is actively critiquing its own outputs rather than generating blind text.³⁶

Repository Standardization and Technical Post-Mortem

The GitHub repository represents the final, measurable outcome of the execution blueprint. It must reflect the rigorous discipline and structural organization of an advanced Machine Learning Engineer.

The repository architecture must be strictly compartmentalized. Raw datasets must never be committed to version control. The /data directory must contain solely the extraction and serialization scripts utilized for the SAP SALT and Bitext transformations. The /models directory must house the LoRA adapter saving mechanisms, the weight merging scripts, and the custom inference wrappers. The /eval directory must contain the benchmarking scripts used to measure TPS latency and retrieval accuracy. A highly specific requirements.txt file must enforce strict version pinning of all libraries to guarantee exact reproducibility across varying compute environments.

The README.md file must be approached not as a basic tutorial, but as a comprehensive engineering post-mortem. It must clearly define the initial hardware bottlenecks or factual hallucination problems, detail the mathematical intuition behind the chosen architectural solutions, and explicitly explain the complex data engineering required to flatten multi-table relational databases into causal language modeling formats.

Finally, the documentation must present empirical evidence of success through clearly formatted benchmark matrices. The practitioner is expected to synthesize their results into a structure that mirrors the following format:

Metric Evaluated	Baseline Architecture	Optimized System (Speculative Decoding/Self-RAG)	Delta / Improvement Factor
Inference Latency (TPS)	14.2 tokens/sec	36.5 tokens/sec	~2.5X Speedup

Enterprise Factual Accuracy	45.8%	88.2%	+42.4%
GPU VRAM Utilization	14.1 GB	14.8 GB	+0.7 GB (Draft Model Overhead)

The execution of these protocols is not a linear progression; the rigorous engineering discipline developed during the manual construction of the complex inference algorithms in Phase 1 directly informs the debugging methodology required for the advanced training loops in Phase 2. The practitioner is strictly directed to commence Day 1 tasks immediately. The selection of the primary research architecture must be finalized, the compute environment secured, and the algorithmic deconstruction of the mathematical formulas into raw Python logic must begin. Precision will evolve through iteration; the immediate mandate is the construction of the baseline system.

Works cited

1. Self-RAG: Learning to Retrieve, Generate and Critique through Self ..., accessed on April 4, 2026, <https://selfrag.github.io/>
2. Self-Rag: Self-reflective Retrieval augmented Generation - arXiv, accessed on April 4, 2026, <https://arxiv.org/html/2310.11511v1>
3. selfrag - Hugging Face, accessed on April 4, 2026, <https://huggingface.co/selfrag>
4. selfrag/selfrag_train_data · Datasets at Hugging Face, accessed on April 4, 2026, https://huggingface.co/datasets/selfrag/selfrag_train_data
5. arXiv:2310.11511v1 [cs.CL] 17 Oct 2023, accessed on April 4, 2026, <https://arxiv.org/pdf/2310.11511>
6. SELF-RAG: LEARNING TO RETRIEVE, GENERATE, AND CRITIQUE THROUGH SELF-REFLECTION - OpenReview, accessed on April 4, 2026, <https://openreview.net/pdf?id=hSyW5go0v8>
7. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection - arXiv, accessed on April 4, 2026, <https://arxiv.org/abs/2310.11511>
8. qiaojin/PubMedQA · Datasets at Hugging Face, accessed on April 4, 2026, <https://huggingface.co/datasets/qiaojin/PubMedQA>
9. Fast Inference from Transformers via Speculative Decoding - Tel Aviv University, accessed on April 4, 2026, <https://cris.tau.ac.il/en/publications/fast-inference-from-transformers-via-speculative-decoding/>
10. arXiv:2211.17192v2 [cs.LG] 18 May 2023, accessed on April 4, 2026, <https://arxiv.org/pdf/2211.17192>
11. Looking back at speculative decoding - Google Research, accessed on April 4, 2026, <https://research.google/blog/looking-back-at-speculative-decoding/>

12. The Machine Learning Practitioner's Guide to Speculative Decoding, accessed on April 4, 2026, <https://machinelearningmastery.com/the-machine-learning-practitioners-guide-to-o-speculative-decoding/>
13. GitHub - mscheong01/speculative_decoding.c: minimal C implementation of speculative decoding based on llama2.c, accessed on April 4, 2026, https://github.com/mscheong01/speculative_decoding.c
14. Speculative Decoding: A Guide With Implementation Examples - DataCamp, accessed on April 4, 2026, <https://www.datacamp.com/tutorial/speculative-decoding>
15. Speculative Decoding in vLLM: Complete Guide to Faster LLM Inference | Jarvislabs.ai Docs, accessed on April 4, 2026, <https://docs.jarvislabs.ai/blog/speculative-decoding-vllm-faster-llm-inference>
16. Speculative Decoding with vLLM — NVIDIA Triton Inference Server, accessed on April 4, 2026, https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tutorials/Feature_Guide/Speculative_Decoding/vLLM/README.html
17. [2211.17192] Fast Inference from Transformers via Speculative Decoding - arXiv, accessed on April 4, 2026, <https://arxiv.org/abs/2211.17192>
18. Better RAG with HyDE - Hypothetical Document Embeddings - Zilliz Learn, accessed on April 4, 2026, <https://zilliz.com/learn/improve-rag-and-information-retrieval-with-hyde-hypothetical-document-embeddings>
19. HyDE for RAG Explained: How Hypothetical Document Embeddings Boost Retrieval Accuracy - Machine Learning Plus, accessed on April 4, 2026, <https://machinelearningplus.com/gen-ai/hypothetical-document-embedding-hyde-a-smarter-rag-method-to-search-documents/>
20. Precise Zero-Shot Dense Retrieval without Relevance Labels, accessed on April 4, 2026, <https://arxiv.org/abs/2212.10496>
21. Precise Zero-Shot Dense Retrieval without Relevance Labels - ACL Anthology, accessed on April 4, 2026, <https://aclanthology.org/2023.acl-long.99/>
22. Hypothetical Document Embeddings (HyDE) - Haystack Documentation, accessed on April 4, 2026, <https://docs.haystack.deepset.ai/docs/hypothetical-document-embeddings-hyde>
23. ERP AI Chatbots: Business Use Cases & Best Practice (2026) - HSO, accessed on April 4, 2026, <https://www.hso.com/blog/erp-ai-chatbots>
24. AI in ERP: Next-Gen ERP Solutions - NetSuite, accessed on April 4, 2026, <https://www.netsuite.com/portal/resource/articles/erp/ai-erp.shtml>
25. bitext/Mistral-7B-Customer-Support - Hugging Face, accessed on April 4, 2026, <https://huggingface.co/bitext/Mistral-7B-Customer-Support>
26. SAP Publishes First Real ERP Dataset to Advance Enterprise AI Research, accessed on April 4, 2026, <https://news.sap.com/2025/04/sap-salt-real-erp-dataset-enterprise-ai-research/>
27. SALT: Sales Autocompletion Linked Business Tables Dataset - arXiv, accessed on April 4, 2026, <https://arxiv.org/html/2501.03413v1>

28. AI-Driven Sales Order Autocompletion in SAP SD: Leveraging the SALT Dataset for Sustainable Supply Chain Optimization - TechRxiv, accessed on April 4, 2026, <https://www.techrxiv.org/users/937938/articles/1308168-ai-driven-sales-order-autocompletion-in-sap-sd-leveraging-the-salt-dataset-for-sustainable-supply-chain-optimization>
29. SAP: Advancing enterprise AI research with first real ERP dataset - Silicon Saxony, accessed on April 4, 2026, <https://silicon-saxony.de/en/sap-advancing-enterprise-ai-research-with-first-real-erp-dataset/>
30. Retail Ecommerce QA Pairs for LLM Conversational Fine-Tuning - Databricks Marketplace, accessed on April 4, 2026, https://marketplace.databricks.com/details/e190708d-19ae-4ddb-aff3-c5aa56a87da4/Bitext-Innovation-International_Retail-Ecommerce-QA-Pairs-for-LLM-Conv-ersational-FineTuning
31. Multi-Turn Conversational Data Fine-Tuning - Emergent Mind, accessed on April 4, 2026, <https://www.emergentmind.com/topics/multi-turn-conversational-data-fine-tuning>
32. Dataset formats and types - Hugging Face, accessed on April 4, 2026, https://huggingface.co/docs/trl/dataset_formats
33. Fine-Tuning LLMs for Multi-Turn Conversations: A Technical Deep Dive - Together AI, accessed on April 4, 2026, <https://www.together.ai/blog/fine-tuning-llms-for-multi-turn-conversations-a-technical-deep-dive>
34. SAP/SALT · Datasets at Hugging Face, accessed on April 4, 2026, <https://huggingface.co/datasets/SAP/SALT>
35. SAP-samples/salt: Source code and data for the paper "SALT: Sales Autocompletion Linked Business Tables Dataset" - GitHub, accessed on April 4, 2026, <https://github.com/SAP-samples/salt>
36. selfrag/selfrag_llama2_7b - Hugging Face, accessed on April 4, 2026, https://huggingface.co/selfrag/selfrag_llama2_7b
37. Build a self-RAG agent with IBM Granite LLMs: A practical guide, accessed on April 4, 2026, <https://www.ibm.com/think/tutorials/build-self-rag-agent-langgraph-granite>